Introduction to Deep Reinforcement Learning

Notes by Marco Galletti
Lecturer: Georgina Nouli

Technical University of Munich
Winter Semester 2026/27

Contents

1 Search Algorithms **2**
1.1 Search problems 2
1.2 Uninformed and informed search 3
1.3 Adversarial and expectimax search 8

1 Search Algorithms

Decision making is first studied from the optimisation side: a deterministic problem is modelled as a graph and solved by searching it for a (least-cost) path to a goal; at the end, an adversary and then randomness are added.

1.1 Search problems

Definition 1.1 (Algorithm, completeness, optimality)**.** An **algorithm** is a finite sequence of well-defined, computer-implementable instructions that solves a specific set of computable problems: it takes a finite input, processes it unambiguously at each operation and returns its output within a finite amount of time. An algorithm is **complete** if, on any input for which a solution exists, it finds one, and **optimal** if it is guaranteed to find the best feasible solution. A **heuristic** is an algorithm that is not guaranteed to be optimal.

Definition 1.2 (Graphs, paths, trees)**.** A **graph** consists of **vertices** (**nodes**), representing objects, and **edges**, representing relations between them; edges may be directed or undirected, **arcs** are always directed, and **weights** on the edges describe the relations (e.g. costs). A **path** is a sequence of nodes in which consecutive nodes are joined by an edge. A **shortest path** between two nodes is one with the fewest edges (actions), a **least-cost path** one with the least sum of weights; they coincide when all weights are 1. A **cycle** is a non-empty path whose only repeated vertices are the first and the last. A **root** is a vertex with only outgoing edges, and a **rooted graph** is a graph in which one vertex is the root. A **tree** is a cycle-free rooted graph with exactly one path from the root to each vertex.

For instance, if a start node is joined to a goal directly with weight 6 and through a node A with weights 1 and 2, the shortest path is the direct edge (one action), the least-cost path goes through A (cost 3).

Definition 1.3 (Search problem)**.** A **search problem** consists of a **state space** S with **states** $s \in S$; for each state an **action space** $A(s)$ with **actions** $a \in A(s)$; a **successor function** T, giving the state $T(s, a) \in S$ reached by taking $a \in A(s)$ in s, together with the cost of the action; costs are nonnegative; a **start state** $s_0 \in S$; and a **goal test**, which tells whether a state is a goal. A **solution** (or **plan**) is a sequence of actions $\pi = (a_1, \dots, a_k)$ that transforms the start state into a goal:

$$s_i = T(s_{i-1}, a_i), \quad a_i \in A(s_{i-1}) \quad (i = 1, \dots, k), \qquad s_k \text{ is a goal.}$$

Its cost is the sum of the costs of its actions; a least-cost solution is denoted π^*.

Example 1.4. In a graph with states $S, A, \dots, G$, start S and goal G, let the action $\mathrm{go}(X)$ move along an edge to X (so $T(S, \mathrm{go}(C)) = C$). If the graph has the edges $S \to C$ and $C \to G$, then $(\mathrm{go}(C), \mathrm{go}(G))$ is a solution. Likewise, travelling from Munich to Berlin has cities as states and trips to another city as actions, and a solution is a travel plan; in Pac-Man a state is the position of Pac-Man, the positions of the monsters and the subset of remaining food dots, the actions are North, South, East, West.

State space graphs and search trees. A search problem can be represented in two ways. In the **state space graph** the nodes are the states, the root is the start state, the arcs are the transitions to successors (actions), the goal test is a set of goal nodes (possibly one) and a solution is a path in the graph: *each state occurs only once.* In the **search tree** the root is the start state, the children of a node are its successors, the goal nodes are leaves and a solution is a path in the tree: *a state can occur several times*, once for every path that reaches it (from Munich, Berlin can be reached through Leipzig, through Nuremberg and Leipzig, through Hanover, …). For large problems neither can be built completely in an efficient way.

Definition 1.5 (Tree search, graph search)**.** A search algorithm explores a search graph or tree systematically, starting from the start state. **Expanding** ("visiting") a node means generating its successors. The nodes generated but not yet expanded form the **frontier**, and the data structure of the frontier (queue, stack, …) determines the order of exploration. **Tree search** may expand a state several times. **Graph search** expands each state at most once: it is tree search plus a **marked list** of the states already expanded.

The representation of the problem and the algorithm are independent of each other, and any combination is possible: a search tree can also be explored by graph search. For instance, if S leads to A, B and C, each of these leads to D, and then $D \to E \to G$, the search tree contains D and E three times; tree search may expand them three times, while graph search expands them once. An optimal search algorithm finds least-cost plans.

1.2 Uninformed and informed search

Uninformed search uses only the problem definition. All algorithms below are run as graph search on the graph of fig. 1, start S, goal G, with children taken in alphabetical order. Depth-first and breadth-first search ignore the weights. In the traces the frontier is listed in the order in which its nodes will be removed. The pseudocode stops at the goal; the returned path is the path to the goal in the search tree, read backwards from the goal through the node each node was generated from.

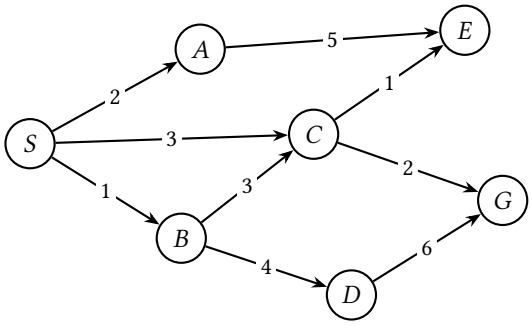


Figure 1: Example graph for uninformed search.

Depth-first search (DFS). DFS explores the children of the active node first: its strategy is to **expand a deepest node first**. Recursively:

Algorithm 1 Depth-first (graph) search, called as DFS(G, s_0)

```
1: procedure DFS(G, v)                          ▷ G: search graph, v: current node
2:     mark v
3:     if v is a goal then terminate
4:     end if
5:     for each child c of v in G do
6:         if c is not marked then DFS(G, c)
7:         end if
8:     end for
9: end procedure
```

When a node has no unmarked children and the search has not terminated, the recursion returns to the caller, i.e. to the direct predecessor in the graph. Equivalently, the frontier is a **LIFO stack**: pop the top node; if it is a goal, stop; otherwise, if it is not marked, mark it and push its unmarked children, the alphabetically first one last, so that it is on top. On fig. 1 (table 1) the stack holds C twice after expanding

B, since C was pushed both by S and by B; the recursive version marks the same states in the same order S, A, E, B, C, G. DFS returns $S \to B \to C \to G$ (three actions).

Popped	Stack after the step (top first)	Marked
S	A, B, C	S
A	E, B, C	S, A
E	B, C	S, A, E
B	C, D, C	S, A, E, B
C	G, D, C	S, A, E, B, C
G	goal: return $S \to B \to C \to G$	

Table 1: DFS with a stack on fig. 1.

Breadth-first search (BFS). BFS explores the neighbours first: its strategy is to **expand the shallowest node first**, with the frontier a **FIFO queue**. The operation *dequeue* returns and removes the first element of the queue, *enqueue* inserts an element at its end, so the node that has been in the queue longest is expanded first.

Algorithm 2 Breadth-first (graph) search

Require: search graph G, start node s_0
if s_0 is a goal **then terminate**
end if
$Q \leftarrow$ queue containing s_0
while Q is not empty **do**
    $v \leftarrow Q$.dequeue()
    if v is not marked **then**
        mark v
        for each child c of v in G **do**
            if c is a goal **then terminate**
            end if
            Q.enqueue(c)
        end for
    end if
end while

BFS tests a node for the goal when it is generated. On fig. 1 (table 2) it returns $S \to C \to G$, two actions.

Dequeued	Queue after the step (front first)	Marked
S	A, B, C	S
A	B, C, E	S, A
B	C, E, C, D	S, A, B
C	E, C, D, E; child G is a goal: return $S \to C \to G$	S, A, B, C

Table 2: BFS on fig. 1.

Uniform-cost search (UCS). BFS finds a shortest path (fewest actions), but not necessarily a least-cost path.

Example 1.6. Let $S \to A$ cost 1, $A \to B$ cost 2, $B \to G$ cost 1 and $S \to G$ cost 5. BFS generates G as a child of S and returns $S \to G$ with cost 5, while the best solution $S \to A \to B \to G$ costs $1 + 2 + 1 = 4$; UCS finds the latter.

UCS explores the cheapest nodes first: its strategy is to **expand the cheapest node first**, i.e. the one with the least cumulative path cost $g(v)$ from the start, and its frontier is a **priority queue** ordered by g. The operation $replace(c, k)$ replaces the cost stored for c in the queue by k.

Algorithm 3 Uniform-cost (graph) search

Require: search graph G with weights w, start node s_0

$Q \leftarrow$ priority queue containing s_0 with $g(s_0) = 0$
while Q is not empty **do**
  $v \leftarrow Q$.pop() ▷ node of least g
  if v is a goal **then terminate**
  end if
  mark v
  for each child c of v in G **do**
    $g' \leftarrow g(v) + w(v, c)$ ▷ cost to c through v
    if c is not marked and not in Q **then**
      Q.insert(c, g')
    else if c is in Q with $g(c) > g'$ **then**
      Q.replace(c, g')
    end if
  end for
end while

Unlike BFS, UCS tests for the goal when a node is popped, not when it is generated; this is what makes it find a least-cost path. After a replace, the predecessor of c on its current cheapest path is v. On fig. 1 (table 3, ties broken alphabetically) it returns $S \to C \to G$ with cost 5.

Popped (g)	Priority queue after the step, $c\,(g)$	Marked
$S\,(0)$	$B\,(1), A\,(2), C\,(3)$	S
$B\,(1)$	$A\,(2), C\,(3), D\,(5)$ (C through B: $4 > 3$, kept)	S, B
$A\,(2)$	$C\,(3), D\,(5), E\,(7)$	S, B, A
$C\,(3)$	$E\,(4), D\,(5), G\,(5)$ (E through C: $4 < 7$, replaced)	S, B, A, C
$E\,(4)$	$D\,(5), G\,(5)$	S, B, A, C, E
$D\,(5)$	$G\,(5)$ (G through D: $11 > 5$, kept)	S, B, A, C, E, D
$G\,(5)$	goal: return $S \to C \to G$, cost 5	

Table 3: UCS on fig. 1.

Informed search. **Informed search** adds a **distance approximation** $h(n)$ to the search: a function estimating the distance from a state n to a goal. It depends on the problem: for shortest path problems, e.g., the Manhattan or the Euclidean distance; for puzzle or ordering problems, the number of misplaced items. For the route from Munich to Berlin, h can be the straight-line distance, 504 km, while the least-cost road distance is 584 km. The approximation h itself is also called a **heuristic**.

Greedy search. **Greedy search** expands the frontier node with the least approximated distance $h(n)$ to a goal, i.e. the one assumed to be closest to a goal. It always makes a locally optimal choice in order to approximate a globally optimal solution, but it is a heuristic (definition 1.1): it is not guaranteed to find an optimal solution, it can lead straight to the wrong goal, and in the worst case it behaves like a

badly guided DFS. On the tree of fig. 2 it expands S, then 2 ($h = 13 < 25$), 6 ($h = 5$), 5 ($h = 6$), and finally pops G ($h = 0$) (table 4).

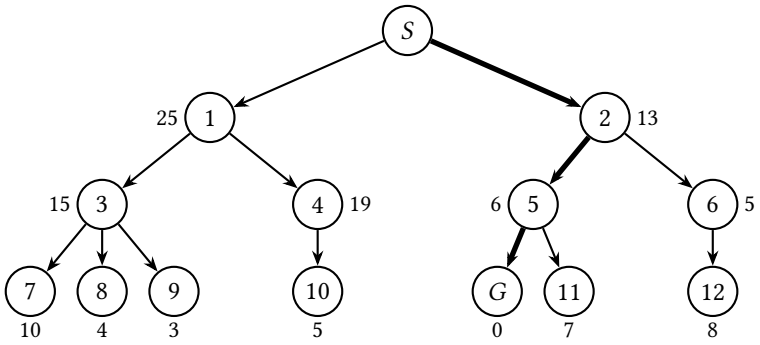


Figure 2: Greedy search example.

Expanded	Frontier after the step, $n\,(h)$
S	2 (13), 1 (25)
2	6 (5), 5 (6), 1 (25)
6	5 (6), 12 (8), 1 (25)
5	G (0), 11 (7), 12 (8), 1 (25)
G	goal: return $S \to 2 \to 5 \to G$

Table 4: Greedy search on fig. 2.

A* search. A* combines UCS and greedy search. UCS orders the frontier by the path cost $g(n)$ from the start to n (the actual "backward" cost), greedy search by the estimated "forward" cost $h(n)$ from n to a goal; **A* search** expands the frontier node with the least

$$f(n) = g(n) + h(n),$$

the estimated cost of a path from the start through n to a goal. As in UCS, the goal test is applied when a node is popped. On fig. 3 (table 5) A* returns $S \to A \to B \to G$ with cost 5, the least cost. Ordering by g alone, UCS would also expand E and F, cheap to reach but with no path to the goal, before popping G; ordering by h alone, greedy search would go from A to C, which has the smallest estimate $h(C) = 1$ but costs 6 to reach.

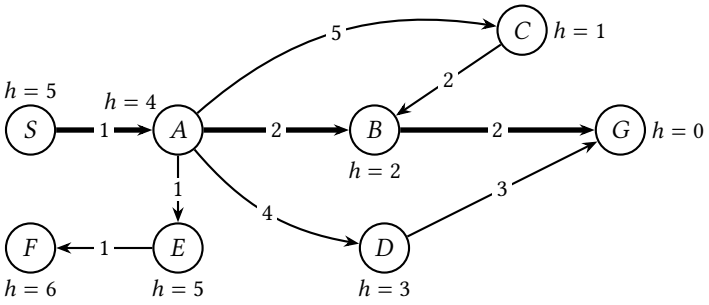


Figure 3: A* search example.

Expanded	Frontier after the step, $n : \ g + h = f$
S	$A : \ 1 + 4 = 5$
A	$B : \ 3 + 2 = 5, \quad C : \ 6 + 1 = 7, \quad E : \ 2 + 5 = 7, \quad D : \ 5 + 3 = 8$
B	$G : \ 5 + 0 = 5, \quad C : \ 7, \quad E : \ 7, \quad D : \ 8$
G	goal: return $S \to A \to B \to G$, cost 5

Table 5: A* search on fig. 3.

Admissibility. A* need not be optimal.

Example 1.7. Let $S \to A$ cost 1, $A \to G$ cost 3 and $S \to G$ cost 5, with $h(S) = 7$, $h(A) = 6$, $h(G) = 0$. After expanding S the frontier holds G with $f(G) = 5 + 0 = 5$ and A with $f(A) = 1 + 6 = 7$, so A* pops G and returns $S \to G$ with cost 5; it never expands A, although $S \to A \to G$ costs 4. The problem is that the estimate $h(A) = 6$ exceeds the true remaining cost 3 on the optimal path, so that $f(A)$ exceeds the actual cost 5 of the non-optimal path.

Definition 1.8 (Admissible heuristic)**.** Let $h^*(n)$ be the true cost from n to a nearest goal. A distance approximation h is **admissible** (**optimistic**) if

$$0 \le h(n) \le h^*(n) \qquad \text{for all nodes } n,$$

and **inadmissible** (**pessimistic**) otherwise.

The straight-line distance is admissible, since no road is shorter than the straight line ($504 \le 584$ above). In example 1.7, $h(A) = 6 > h^*(A) = 3$; the heuristic of fig. 3 is admissible ($h^*(C) = 4 \ge 1$, and $h = h^*$ at S, A, B, D, G). In practice, finding an admissible distance measure is often challenging.

Theorem 1.9 (Optimality of A* tree search)**.** *If h is admissible, A* organised as tree search is optimal.*

Proof. Let p^* be an optimal solution path, with cost c^*, ending in the goal G_1, and let G_2 be a suboptimal goal on the frontier, reached by a path of cost $g(G_2) > c^*$. We show that every node N of p^* (an ancestor of G_1) is expanded before G_2, so that the suboptimal path cannot become the solution. While G_1 has not been expanded, some node N of p^* is on the frontier, since the start node is on p^* and expanding a node of p^* puts its successor on p^* on the frontier.

1. $f(N) \le f(G_1)$: by admissibility $h(N) \le h^*(N)$, which is at most the cost from N to G_1 along p^*, so $f(N) = g(N) + h(N) \le c^* = g(G_1)$; and $g(G_1) = f(G_1)$ because $h = 0$ at a goal.
2. $f(G_1) < f(G_2)$: G_2 is suboptimal, so $g(G_1) < g(G_2)$, and again $h = 0$ at both goals.
3. Hence $f(N) \le f(G_1) < f(G_2)$, and N is expanded before G_2.

Applying this to every node of p^*, all ancestors of G_1 are expanded before G_2, and then G_1 itself is popped before G_2 since $f(G_1) < f(G_2)$. So A* returns the optimal path. □

UCS expands equally in all "directions", in contours of increasing g around the start; A* expands mainly towards the goal, but hedges its bets enough to ensure optimality.

Algorithm	Expands first	Frontier	Guarantee
DFS	deepest node	LIFO stack	not optimal (table 1)
BFS	shallowest node	FIFO queue	shortest path (fewest actions)
UCS	least $g(n)$	priority queue	least-cost path
Greedy	least $h(n)$	priority queue	none (heuristic)
A*	least $f(n) = g(n) + h(n)$	priority queue	least-cost path if h admissible (tree search)

Table 6: Search algorithms compared.

1.3 Adversarial and expectimax search

Agents and utilities. An **agent** is any system that perceives its environment and takes actions within it; it uses prior knowledge, past experience, goals and observations, and receives feedback (outcomes) from the environment. An agent always maximises its utility.

Definition 1.10 (Utility, lottery)**.** A **utility function** U maps outcomes (states of the environment) $s \in S$ to real numbers describing the agent's satisfaction with them. A situation is **deterministic** if it has only one possible, predefined outcome, and a **lottery** if one of several outcomes is drawn at random; a lottery with outcomes x_i and probabilities p_i is written

$$L = [p_1, x_1;\ p_2, x_2;\ \ldots;\ p_n, x_n].$$

A utility satisfies

$$U(s_1) \geq U(s_2) \iff s_1 \succeq s_2, \qquad U([p_1, s_1; \ldots; p_n, s_n]) = \sum_i p_i\, U(s_i).$$

The agent prefers ($\succeq$) a state of higher utility, and the utility of a lottery is its **expected utility**, the sum of the utilities weighted by their probabilities. A **rational agent** chooses, based on its knowledge, the action that maximises its expected utility.

For instance, flipping a coin is the lottery $[0.5, \text{head};\ 0.5, \text{tail}]$. Utilities represent the agent's reward (e.g. $+1$ for winning a game and -1 for losing), and any "rational" preferences can be summarised by a utility function. A **positive affine transformation** of a utility is again a utility, representing the same preferences:

$$U'(s) = k_1 U(s) + k_2, \qquad k_1 > 0, \tag{1}$$

since U' orders states like U, and $\sum_i p_i U'(s_i) = k_1 \sum_i p_i U(s_i) + k_2$ orders lotteries like U.

Adversarial games. In **zero-sum games** the agents have opposite utilities (values on outcomes): agent A wants to maximise the objective value and agent B to minimise it, so they compete against each other. In **non-zero-sum games** the agents have independent utilities, and other scenarios are possible, e.g. cooperation (both maximise the same objective) or indifference (the objectives are unrelated). Here two players, **MAX** and **MIN**, play a zero-sum game, alternating turns (tic-tac-toe, chess, checkers).

Definition 1.11 (Game tree)**.** A two-player game is modelled by a search tree whose nodes are the states of the game, $S = S^{\text{MAX}} \cup S^{\text{MIN}} \cup S^T$ (states where MAX or MIN moves, and terminal states), with the start of the game as root. Arcs are the actions of the player in turn:

$$a \in A^{\text{MAX}},\ s \xrightarrow{a} s' \ \Rightarrow\ s \in S^{\text{MAX}},\ s' \in S^{\text{MIN}} \cup S^T; \qquad a \in A^{\text{MIN}},\ s \xrightarrow{a} s' \ \Rightarrow\ s \in S^{\text{MIN}},\ s' \in S^{\text{MAX}} \cup S^T.$$

A node is a goal (terminal) node when the game is over, e.g. in tic-tac-toe when a player wins or no free cell is left (draw), and a **utility function** gives the value of each terminal state (-1, 0, $+1$ for loss, draw, win of MAX). A solution is a **strategy**: the best action in each state.

Definition 1.12 (Minimax value)**.** The **value** $V(s)$ of a state is the best achievable outcome (utility) from s. The **minimax value** is the best utility the player in turn can achieve against a rational (optimal) adversary who moves in every second layer:

$$V(s) = \begin{cases} U(s), & s \in S^T \text{ (known)}, \\ \max_{s' \in \text{successors}(s)} V(s'), & s \in S^{\text{MAX}} \text{ (agent's control)}, \\ \min_{s' \in \text{successors}(s)} V(s'), & s \in S^{\text{MIN}} \text{ (opponent's control)}. \end{cases}$$

Minimax search generates the complete game tree (all playing sequences) with DFS, computes the utility of each terminal state, propagates the values up the tree (minimum of the successors at MIN nodes, maximum at MAX nodes), and from the initial state MAX chooses the move to the successor with the highest value. In fig. 4(a) the MIN nodes get $\min(8,2) = 2$ and $\min(5,6) = 5$, and the root $\max(2,5) = 5$: MAX moves right.

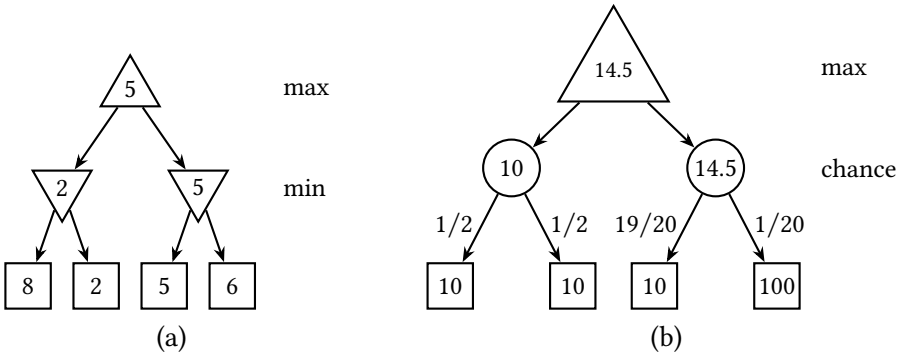


Figure 4: (a) Minimax tree; (b) expectimax tree.

Since minimax is DFS, it has time complexity $O(b^m)$ and space complexity $O(b\,m)$, with b the maximal number of successors and m the depth (number of layers) of the tree. For chess $b \approx 35$ and $m \approx 100$, so the exact solution is far too expensive: a chess program needs about 100 seconds to explore 8 layers.

Depth-limited search. When the tree is too big to explore completely, **depth-limited search** runs DFS only to a limited depth and applies an **evaluation function** to the non-terminal positions there, which act as new terminal states. This loses the guarantee of optimal play; the more layers are considered, the closer the strategy may get to the optimal one. For instance, if the evaluation function gives $-1, -2$ below the left MIN node and $4, 9$ below the right one, the MIN nodes get -2 and 4, and the root 4.

Expectimax search. Outcomes can be uncertain because of explicit randomness (rolling dice), unpredictable opponents (the ghosts in Pac-Man respond randomly) or actions that fail (the wheels of a robot slip). **Expectimax search** then computes the average score under optimal play instead of the worst case: besides MAX nodes the tree has **chance nodes**, whose value is the expected utility of their children, the probability-weighted average; a rational agent picks the action of maximal expected utility. In fig. 4(b) the two actions lead to the lotteries $[\frac{1}{2}, 10;\ \frac{1}{2}, 10]$ and $[\frac{19}{20}, 10;\ \frac{1}{20}, 100]$, with values

$$\tfrac{1}{2}\cdot 10 + \tfrac{1}{2}\cdot 10 = 10, \qquad \tfrac{19}{20}\cdot 10 + \tfrac{1}{20}\cdot 100 = 9.5 + 5 = 14.5,$$

so MAX takes the right action and the root value is 14.5. This uncertainty will be formalised by Markov decision processes in the next deck.

Transformations of utilities. For minimax only the ordering of the terminal values matters, not their scale: more valuable states must have higher utilities, and any increasing transformation φ leaves the decisions unchanged, since $\varphi(\max(x,y)) = \max(\varphi(x), \varphi(y))$ and likewise for min (**insensitivity to monotonic transformations**). For expectimax the scale and magnitudes matter, because they change the ordering of the weighted sums.

Example 1.13. MAX chooses between the lotteries $[0.4, 0;\ 0.6, 40]$ and $[0.5, 20;\ 0.5, 30]$. Their expected utilities are $0.4 \cdot 0 + 0.6 \cdot 40 = 24 < 25 = 0.5 \cdot 20 + 0.5 \cdot 30$, so expectimax takes the second. After the transformation $x \mapsto x^2$, increasing on these nonnegative utilities, the lotteries become $[0.4, 0;\ 0.6, 1600]$ and $[0.5, 400;\ 0.5, 900]$, with $0.6 \cdot 1600 = 960 > 650 = 0.5 \cdot 400 + 0.5 \cdot 900$, and expectimax takes the first. Minimax would value both lotteries by their worst outcome and choose the same action before and after

the transformation. Only positive affine transformations (1) are guaranteed to preserve the expectimax decision.